

033 动量学习率与调度实验

本实验要回答三个可以实际核对的问题：动量更新是否与手算一致，学习率是否按真正的参数更新次数推进，恢复训练是否仍走原来的轨迹。请先读正文第4节，再打开Notebook。所有计算仅需CPU，无数据或预训练模型下载。

1 先确认环境与文件

已验证环境为Python3.12.14、NumPy2.3.5、PyTorch2.7.1+cpu，单线程float64。请用已有兼容环境，或按README中的隔离环境说明安装。environment.yml是可复建声明，不表示在全新Anaconda、每个操作系统或GPU上重新验证过。实际交付Notebook在新的Python进程中用IPython InProcessKernel从头顺序执行；没有把浏览器界面或socket传输说成已测试。

本讲不依赖相邻单元的代码。将本讲整个目录放在一起，在本讲目录运行：

```
python experiment.py --output outputs
python -m unittest -v test_experiment.py
python -O experiment.py --output optimized_outputs
```

脚本根据自身位置寻找data，终端工作目录不影响脚本读入；Notebook应在本讲目录打开。输出目录名称可改，不能指向源码根目录、data或figures。参考输入字节发生变化时固定入口会拒绝运行，这是防止新数据与旧结论混用，不是阻止你探索。探索请调用公开函数并写入独立目录。

文件	检查内容
data/hand-example.json	两样本、9参数、两次学习率与动量
data/samples.csv	48行合成网格，ID为0至47
data/config.json	12epoch、4样本微批次、累积2次
outputs/trace.json	两次完整前向、局部导数、路径贡献、全梯度、缓冲与新前向
outputs/network.json	四方案逐步loss、LR、完整状态及排列
outputs/schedules.csv	正确学习率与两倍时钟错误示意
outputs/checkpoint-epoch-04.json	完成24次更新时的检查点
outputs/final-state.json	完成原定72次更新时的完整状态

其他输出包括quadratic.json、resume.json和summary.json。Notebook重新计算到notebook_outputs，并逐文件与这8份参考文件比较。输出比对适用于本次验证环境；跨平台、版本或后端可能存在尾数差异，应先检查原因和数值容差，不要简单改摘要假装原图仍适用。

2 把反向传播接到真正的更新

在Notebook中依次检查第一次前向的X、Z、H、P、残差和loss。此处标签形状必须为(2,1)，不能用(2,)让广播悄悄生成(2,2)。预激活第二行第一列为-0.1，该路径的ReLU导数为0。

先遮住答案，用纸笔计算 $\delta P_1 = -0.28/2 = -0.14$ 、 $\delta H_{12} = (-0.14)(-0.5) = 0.07$ 。再追到两个样本共享的 W_{12} ，把两条路径相加得到0.045。程序中per_sample_parameter_contribution的shape是(2,9)，每列之和才是对应参数的完整梯度。

```
import experiment as e
import numpy as np
e.configure_cpu()
cfg, hand, X, y = e.read_inputs()
trace = e.trace_steps(hand)
s = trace['steps'][0]
contribution = np.array(s['per_sample_parameter_contribution'])
np.testing.assert_allclose(contribution.sum(axis=0), s['gradient'])
print(s['loss'], s['theta_after'], s['next_forward']['loss'])
```

应得到初始loss约0.0221、更新后全部9参数，以及新loss约0.0084925618773。第二次必须用新前向产生新梯度，不能重复第一次梯度。核对 W_{11} 的第二次缓冲-0.121110688512和位移0.0060555344256，再检查第二次更新后的loss约0.00270862158259。

验收要求不是“loss下降”：逐项记录两个样本的全部局部导数、9参数完整梯度、旧缓冲、新缓冲、实际LR及全部更新后值。Notebook保留这些完整输出，不需要读者自己补回省略号。

3 在二次谷里隔离机制

打开quadratic.json和图3。四方案起点都是(3,1)，每个方案执行80次更新。固定SGD与固定动量共享LR0.045，后者的动量为0.8；调度方案峰值仍为0.045、下限0.002，预热方案前12次逐渐升高。

请分别观察x方向和y方向，而不是只看总loss。SGD在陡方向反号，缓方向慢慢缩小；动量会改变这两个方向的递推。保留所有中间点，寻找函数值短暂上升的位置。

```
stable = e.quadratic_run(steps=50, lr=0.045, momentum=0)
unstable = e.quadratic_run(steps=50, lr=0.06, momentum=0)
print(stable[-1]['loss'], unstable[-1]['loss'])
```

预期第二条轨迹会增大，因为陡方向乘数变为 $1 - 0.06 \times 40 = -1.4$ 。这个失败是稳定性条件被破坏，并不需要以反向传播错误来解释。不要把此处0.05的上限当作所有网络的学习率上限。

4 先测试学习率端点 再接入训练

```
for k in [0, 11, 12, 24, 71, 72]:
    print(k, e.schedule_lr(k, 72, 0.06, 0.006, 12))
```

前四个关键含义分别是第一步、预热末步、第一次衰减、恢复后的第一步；71是原预算最后一次，72只说明函数随后保持下限。训练不因此自动增加一次更新。

将CSV中的更新索引、正确预热余弦、错误两倍时钟三列画在同一坐标系。错误示意在真实 $k=36$ 时到达下限，正确方案此时尚未结束衰减。这个图只重采样学习率函数，未宣称运行了错误训练循环。

再检查network.json：每epoch恰有6次更新；每次`lr_used`是optimizer.step之前读到的数，`lr_next`是scheduler.step之后的数。固定SGD和固定动量两条曲线学习率相同，但动量缓冲不同。学习率相同不等于实际位移大小相同。

5 比较小网络时守住预算

四方案均为同一9参数初始化、同一48点、同一epoch排列、同一72次更新。每次更新后对固定48点重新计算loss，读图时可直接比较相同横坐标。

请制作四行实验记录，写出方法、峰值LR、下限LR、动量、warmup长度、更新数、样本呈现数和最终固定数据loss。你应看到本例固定动量终点最低；不能据此推广为“无需调度”，同样不能把预热视为必需。

若想改变超参数，保留原cfg，深拷贝后调用`train_network`，将结果写到新文件。公开API支持1至50个epoch，完整等大小累积窗口，总更新数至少2，动量在 $[0, 0.99]$ ，LR峰值在 $[1e-8, 0.2]$ 。数据shape必须为 $(n, 2)$ 和 $(n, 1)$ ， $1 \leq n \leq 128$ ，非布尔、有限且绝对值 ≤ 100 。这些是有限教学函数的契约，不是PyTorch普遍限制。

6 恢复后逐项核对轨迹

先完成一遍正常实验，再在新终端进程执行：

```
python experiment.py \
  --resume-checkpoint outputs/checkpoint-epoch-04.json \
  --output resumed_outputs
```

上面的反斜线续行适用于Bash/zsh；其他终端可把三行连成一行并去掉续行反斜线。程序校验检查点格式、数据和配置、完成预算、LR及调度计数，然后只继续剩余48次更新。它会同时计算未中断基准，拒绝把不一致结果写成成功。完成72次更新的final-state不是可继续的剩余预算检查点；直接拿它恢复会被拒绝。

```
from pathlib import Path
left = Path('outputs/final-state.json').read_bytes()
right = Path('resumed_outputs/final-state.json').read_bytes()
if left != right:
    raise RuntimeError('完整状态不一致，请先定位差异')
print('最终状态逐字节一致')
```

看resume.json的三个故意漏状态实验。漏动量改变方向；漏采样RNG改变后续排列；漏scheduler有一个更隐蔽的现象：第一步LR正确，第二步才跳回0.01。必须比较后续序列，而不只在加载瞬间打印一次LR。

本检查点是课程自行生成的JSON，摘要仅检测意外修改；不加载任意pickle文件。本实验只恢复完整epoch边界，没有验证累积窗口中途、DataLoader多worker、分布式、GPU或混合精度恢复。

7 测试失败应该留下什么

运行15组测试，记录失败的参数、shape或更新索引。测试包括独立Fraction链式计算、18坐标有限差分、100条10步有理数动量序列、1024个符号EMA序列、二次递推、四方案全程NumPy反向核对、36次不同seed/切点/方案精确恢复，以及错误状态拒绝。

固定图脚本先核对3输入和8结果，Notebook第一格也先完成同样检查，并核对用于展示的图。不要把第一格验证失败删掉后继续运行旧图；新实验应有自己的数据、输出与结论。所有计算、校验和序列化成功后，输出函数才创建目标目录并替换文件。每文件替换是原子的，但一次实验的8文件写入不是多文件断电事务；写入中途的磁盘故障可能留下新旧混合，需要重新运行和核对摘要。

最终提交：完整两步手算表、三种时钟预算、四方案等预算比较、精确恢复证据，以及一个你能够定位原因的失败反例。把可复现的机制结论与尚未验证的泛化或硬件性能分开写。